

Tema 02 — Inserción de Código en Páginas Web

IES Ntra. Sra. de los Remedios (Ubrique) · 2º DAW · Desarrollo Web en Entorno Servidor · Curso 2026/2027

Introducción

El desarrollo de aplicaciones web del lado del servidor se apoya en lenguajes y tecnologías que ejecuta un software especializado, el **servidor web**, instalado en uno o varios equipos.

La elección del lenguaje condiciona:

- La distribución de la lógica de la aplicación.
- La forma de recibir y tratar los datos que envía el cliente.
- La gestión del flujo de trabajo de la aplicación.
- La configuración del entorno del servidor.

En este tema se presentan los distintos tipos de servidores web y las características principales de los lenguajes de servidor que permiten **insertar código en páginas HTML**. Se usa PHP como lenguaje de referencia.

Objetivos

- Reconocer la arquitectura de las aplicaciones web del lado del servidor según el lenguaje utilizado.
- Aprender a generar código de forma dinámica para que lo muestre el cliente web.
- Conocer la sintaxis y las etiquetas propias de cada lenguaje de servidor que permite insertar código en páginas web.
- Dominar la declaración de variables, los tipos de datos simples y la conversión entre ellos.
- Comprender la importancia del ámbito de las variables y su influencia en el desarrollo de aplicaciones web.

Marco normativo y currículo

Este tema se enmarca en el **módulo profesional 0613 «Desarrollo web en entorno servidor»** del ciclo formativo de grado superior de **Técnico Superior en Desarrollo de Aplicaciones Web (DAW)**.

Norma	Qué regula
Real Decreto 686/2010, de 20 de mayo	Establece el título de Técnico Superior en Desarrollo de Aplicaciones Web y sus enseñanzas mínimas (BOE). El módulo 0613 tiene equivalencia de 12 créditos ECTS.
Real Decreto 405/2023, de 29 de mayo	Modifica varios títulos de FP, entre ellos el de DAW, y actualiza la redacción de los resultados de aprendizaje del módulo.
Normativa autonómica	Cada comunidad (en Andalucía, la orden de currículo del ciclo DAW) concreta contenidos, horas y orientaciones pedagógicas.

Los contenidos de este tema trabajan principalmente los resultados de aprendizaje **RA1** y **RA2** del módulo:

Resultado de aprendizaje	Criterios de evaluación relacionados con este tema	Apartado del tema
RA1. Selecciona las arquitecturas y tecnologías de programación web en entorno servidor, analizando sus capacidades y características propias.	a) Modelos de ejecución en servidor y en cliente. c) Mecanismos de ejecución de código en los servidores web. d) Servidores de aplicaciones e integración con servidores web. e) Principales lenguajes y tecnologías de servidor. f) Integración de los lenguajes de marcas con los lenguajes de servidor.	1 y 2
RA2. Escribe sentencias ejecutables por un servidor web reconociendo y aplicando procedimientos de integración del código en lenguajes de marcas.	a) Generación de páginas a partir de lenguajes de marcas con código embebido. b) Tecnologías asociadas. c) Etiquetas para incluir código. d) Sintaxis del lenguaje. e) Sentencias simples y sus efectos en el documento. f) Directivas que modifican el comportamiento predeterminado. g) Tipos de variables y operadores. h) Ámbitos de las variables.	3, 4 y 5

Nota: las referencias normativas deben contrastarse con la versión consolidada vigente en el BOE y con la normativa autonómica aplicable al centro. Los criterios se citan de forma resumida.

1. Lenguajes y tecnologías del servidor

Como se vio en el Tema 1, las tecnologías de servidor se basan en un software especial llamado **servidor web**. Su configuración depende del lenguaje elegido, porque cada lenguaje necesita componentes específicos para ejecutarse.

Para dominar el desarrollo de aplicaciones dinámicas es necesario entender el escenario completo en el que se ejecuta una aplicación web dentro de una arquitectura cliente/servidor.

1.1. Qué hace un servidor web

Un servidor web es un programa cuya misión es **servir documentos HTML** (*HyperText Markup Language*). Estos documentos ofrecen al cliente textos con enlaces, imágenes, formularios, botones y objetos incrustados, como animaciones o reproductores de sonido.

1.2. Secuencia de una comunicación cliente-servidor

El intercambio de datos se hace mediante un protocolo, normalmente **HTTP** (o su versión cifrada, **HTTPS**). Por defecto, el servidor «escucha» las peticiones de los clientes en un puerto: el **80** para HTTP y el **443** para HTTPS.

```

Navegador —(1) ¿IP de www.ejemplo.org?—> Servidor DNS
Navegador <—(2) IP— Servidor DNS
Navegador —(3) Petición HTTP/HTTPS—> Servidor web (ejecuta el código)
  
```

Navegador ←(4) Respuesta con HTML—— Servidor web
Navegador: interpreta el HTML y lo muestra

1. **Resolución DNS.** El navegador pide al servidor DNS la traducción de una URL (por ejemplo, <https://www.ieslosremedios.org>) a una dirección IP.
2. **Petición HTTP.** Con la IP obtenida, el navegador envía la petición al servidor web. Escribir directamente la IP en lugar de la URL daría un resultado equivalente.
3. **Procesamiento.** El servidor ejecuta el código asociado al recurso solicitado y genera la respuesta.
4. **Respuesta y visualización.** El servidor envía el código HTML y el navegador lo interpreta y lo muestra en pantalla.

1.3. El protocolo HTTP

Un **protocolo** es un conjunto de reglas y estándares que gobiernan el intercambio de información entre entidades de una red. Existen muchos, cada uno con un propósito: FTP, POP3, SMTP, etc.

HTTP es el protocolo de la capa de aplicación que se usa en cada transacción web. Sus características principales son:

- **Sin estado (*stateless*).** El servidor trata cada petición como una operación independiente de las anteriores.
- **Conexiones persistentes.** Desde HTTP/1.1 pueden mantenerse abiertas para enviar varios objetos con menos conexiones. En HTTP/1.0 cada petición implicaba una conexión nueva.
- **Versiones modernas.** HTTP/2 y HTTP/3 multiplexan varias peticiones en una misma conexión y mejoran el rendimiento.

Ejemplo (HTTP/1.0): pedir una página HTML con 5 imágenes requiere 6 conexiones: una por la página y cinco por las imágenes. Las conexiones persistentes y la multiplexación de las versiones posteriores reducen este coste.

1.4. Clasificación de los servidores web

Según cómo procesan las peticiones de los clientes, y según las estrategias de optimización de su diseño, los servidores web se clasifican en cuatro tipos.

1.4.1. Servidores basados en procesos

Es la estrategia más antigua. Consigue el paralelismo **duplicando el proceso** de ejecución:

1. El proceso principal espera una nueva conexión (petición HTTP).
2. Al llegar, se duplica creando una copia exacta (***fork***).
3. La copia atiende la petición, mientras el proceso principal sigue escuchando.

Esta estrategia admite optimizaciones, como la técnica **pre-fork**, que Apache incluye desde la versión 1.3. Consiste en lanzar de antemano varias copias del proceso que escucha y mantenerlas en memoria hasta que se necesiten.

Ventajas del pre-fork	Inconvenientes del pre-fork
Fácil de implementar.	Mayor consumo de memoria (cada proceso tiene la suya).
Aísla el procesamiento de una petición respecto a otras.	Menor rendimiento, porque la gestión de procesos es tarea del sistema operativo.

1.4.2. Servidores basados en hilos

Son una alternativa más económica en recursos. El procesamiento de peticiones es básicamente igual que en los servidores de procesos. La diferencia está en el concepto de **hilo** (*thread*) frente al de proceso.

- Crear un hilo es menos costoso que duplicar un proceso completo.
- Los hilos de un mismo proceso **comparten espacio de memoria**, lo que reduce de forma drástica el consumo.

Ventaja	Riesgo
Mejor rendimiento y menor uso de memoria.	Al compartir memoria, si un hilo modifica una variable, el resto de hilos del proceso ven el valor modificado. Esto puede causar problemas de seguridad y de concurrencia.

1.4.3. Servidores dirigidos por eventos

Su novedad es el uso de **sockets**: canales de comunicación que permiten que dos procesos intercambien información, incluso estando en máquinas distintas (en este caso, cliente y servidor).

- Las lecturas y escrituras sobre los sockets son **asíncronas y bidireccionales**.
- Aunque cada petición sea independiente (sin estado), el servidor puede recuperar la información intercambiada con un cliente accediendo a su socket.

Ventaja	Inconveniente
Gran velocidad.	La concurrencia es simulada: hay un único proceso y un único hilo que atiende todas las conexiones gestionadas por los sockets.

1.4.4. Servidores implementados en el núcleo del sistema

La idea es ejecutar el servidor web en el espacio del sistema operativo (**kernel**) y no en el nivel de usuario.

- **Ventaja:** acelera el procesamiento de las peticiones.
- **Peligro:** cualquier incidencia de seguridad o rendimiento se convierte en un problema del sistema operativo y puede dejarlo completamente inoperativo.

Resumen comparativo

Tipo	Paralelismo mediante	Consumo de memoria	Aislamiento	Punto débil
Procesos	Duplicación de procesos (<i>fork</i>)	Alto	Alto	Rendimiento y memoria
Hilos	Hilos dentro de un proceso	Bajo	Bajo (memoria compartida)	Problemas de concurrencia y seguridad
Eventos	Sockets asíncronos	Muy bajo	Medio	Concurrencia simulada (un solo hilo)
En el núcleo	Ejecución dentro del <i>kernel</i>	Bajo	Muy bajo	Un fallo afecta a todo el sistema

1.5. Servidores web más populares

Casi todos los servidores soportan varios lenguajes de servidor. Algunos lo hacen **de forma nativa** (por ejemplo, IIS con ASP) y otros **mediante módulos y extensiones** (por ejemplo, Apache con PHP).

También hay que recordar la diferencia vista en el Tema 1:

- **Lenguajes interpretados** (PHP, ASP clásico).
- **Lenguajes precompilados**, que necesitan un entorno de ejecución específico (ASP.NET y aplicaciones basadas en Java).

El soporte para unos u otros depende de los módulos y extensiones del servidor elegido. Una configuración adecuada del núcleo y de las extensiones permite que el código cumpla los requisitos esperados.

Apache HTTP Server

- Servidor HTTP **multiplataforma**, robusto, que implementa los últimos estándares y protocolos.
- **Gratuito y de código abierto**, con una comunidad amplia y abundante documentación.
- Muy **modular y configurable**: el administrador elige qué módulos están disponibles al compilar o ejecutar el servidor.
- Módulos para acceso a bases de datos, páginas protegidas con contraseña, páginas de error personalizadas, registros de actividad en varios formatos, etc.
- Puede funcionar **basado en procesos o basado en hilos** (módulos MPM: *prefork*, *worker* y *event*).

Microsoft IIS

- *Internet Information Services* (antes *Internet Information Server*): servidor web de Microsoft para alojar cualquier tipo de contenido en línea.
- Da **soporte nativo a ASP** (*Active Server Pages*) y a las tecnologías de la plataforma .NET. Permite añadir módulos para otros lenguajes, como PHP.
- Solo funciona en sistemas **Windows**. Se instala como complemento del sistema operativo, con permisos de administrador. En ediciones de cliente, como Windows Pro, no viene instalado por defecto.
- Como Apache, admite extensiones para **reescritura de URL** (que varias URL sean atendidas por el mismo recurso), despliegue de aplicaciones multimedia, acceso a bases de datos y administración remota.

Oracle iPlanet Web Server (antes Sun Java System Web Server)

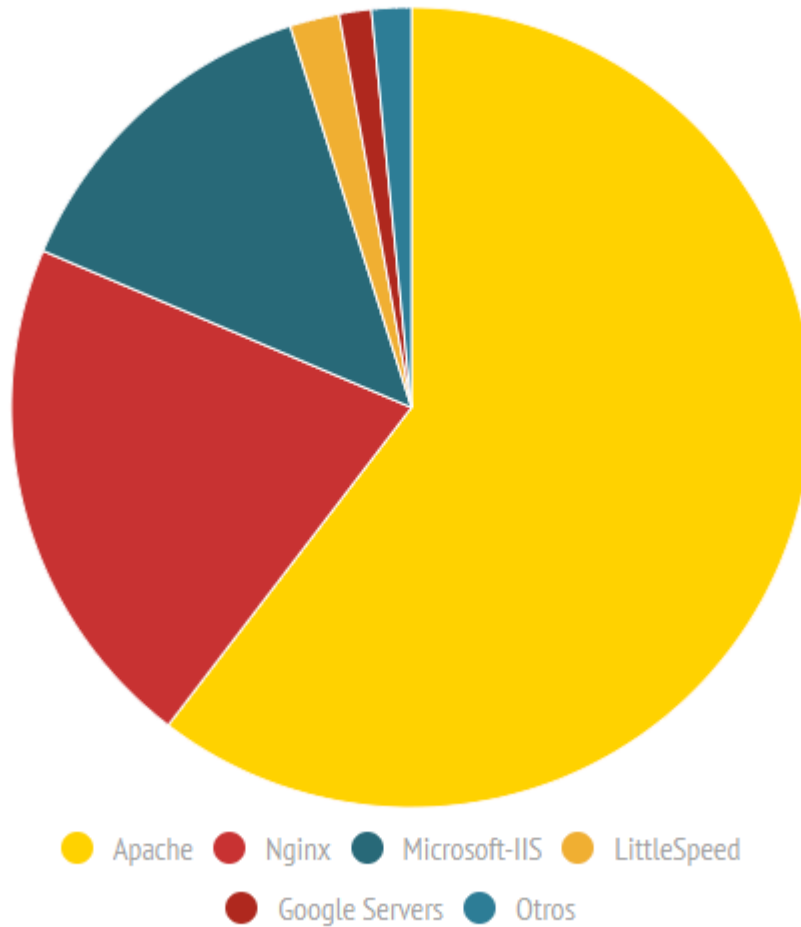
- Servidor de alto rendimiento, escalable y seguro, que sirve contenido estático y dinámico.
- Optimizado para aplicaciones **Java** (JSP, Servlets, NSAPI y CGI).
- Se puede instalar en prácticamente cualquier sistema operativo y admite módulos para PHP, Ruby on Rails, Perl, Python y otros lenguajes.
- Destaca por la virtualización de dominios, la versatilidad de configuración y la seguridad.

Nginx

- Servidor HTTP de **código abierto** con una arquitectura modular de alto rendimiento.
- Ha ganado una gran cuota de mercado desde 2007 y hoy compite con Apache como servidor más usado.
- Es un servidor **asíncrono dirigido por eventos**, conocido por su estabilidad, su amplio conjunto de filtros, su facilidad de configuración y su bajo consumo de recursos.
- Funciona también como **proxy inverso y balanceador de carga**, y como proxy para protocolos como IMAP o POP3.
- Soporta transacciones seguras con SSL/TLS y es multiplataforma.

Lighttpd

Cuota de mercado de Servidores Web



Datos de W3Techs.com

- Servidor web pensado para entornos que exigen **respuestas rápidas**.
- Software libre y de código abierto, bajo licencia **BSD**. Soporta la mayoría de los estándares de comunicación de Internet.
- Bajo consumo de memoria y de CPU en comparación con servidores como Apache.
- Diseñado para soportar un **balanceo de carga elevado** sin perder prestaciones.
- Usa un único proceso con varios hilos, pero no puede crear nuevos hilos según la demanda.

Servidores de aplicaciones (Apache Tomcat)

Los servidores web sirven recursos HTTP, mientras que los **servidores de aplicaciones** ejecutan la lógica de negocio y ofrecen servicios adicionales (gestión de sesiones, transacciones, etc.). **Apache Tomcat** es el ejemplo más conocido para aplicaciones Java (Servlets y JSP). Suele integrarse con un servidor web (Apache o Nginx) que actúa como *proxy* delante de él. Este contenido enlaza con el criterio RA1.d.

Comparativa de servidores web

Servidor	Licencia	Sistema operativo	Soporte de lenguajes	Modelo de ejecución	Punto fuerte
Apache HTTP Server	Libre (Apache 2.0)	Multiplataforma	PHP (módulo/FPM), Perl, Python, otros	Procesos o hilos (MPM)	Flexibilidad y modularidad
Microsoft IIS	Propietaria	Solo Windows	ASP y .NET de forma nativa; PHP por módulo	Procesos y hilos	Integración con el ecosistema Microsoft
Oracle iPlanet	Propietaria (descarga gratuita, soporte de pago)	Multiplataforma	Java (JSP/Servlets); PHP, Perl, Python, otros por módulos	Procesos e hilos	Entornos Java
Nginx	Libre (BSD simplificada)	Multiplataforma	Vía proxy o FastCGI (por ejemplo, PHP-FPM)	Dirigido por eventos	Rendimiento y proxy inverso
Lighttpd	Libre (BSD)	Multiplataforma	Vía FastCGI	Un proceso con hilos	Bajo consumo

La siguiente sección presenta los principios básicos que un programador debe conocer para desarrollar una aplicación ejecutada en cualquiera de estos servidores y adaptar la respuesta a las peticiones del cliente.

2. Obtención del código enviado al cliente

Los usuarios se comunican con el servidor mediante un navegador para obtener una funcionalidad: ver información (textual o multimedia), administrar un sitio, almacenar datos, actualizar contenidos u otras operaciones más complejas.

Para ofrecer toda esa funcionalidad, el servidor debe poder entregar al cliente la información que necesita. De generarla se encarga la **ejecución de código escrito en un lenguaje concreto** por parte del servidor.

2.1. PHP como lenguaje de referencia

PHP (*Hypertext Preprocessor*) es un lenguaje de *scripting* de propósito general y de código abierto. Está diseñado especialmente para el desarrollo web y puede **embeberse** (intercalarse) en código HTML. Los ejemplos de este tema son PHP, y en capítulos sucesivos se mostrarán también ASP y JSP.

Breve historia:

- **1994:** nace a partir de un conjunto de ficheros binarios para recoger datos de tráfico en sitios web.
- **1997:** se reescribe el intérprete (*parser*) para permitir la generación dinámica de contenido y de aplicaciones de servidor.
- **2000:** se publica la versión 4.
- **Actualidad:** PHP sigue en evolución (las versiones 8.x incorporan tipado más estricto, mejoras de rendimiento y nuevas características del lenguaje). Su éxito se debe, en parte, a su relativa facilidad de

aprendizaje.

PHP es un buen ejemplo para comprender la arquitectura de un servidor web e identificar los componentes que intervienen al procesar una petición.

2.2. Arquitectura de PHP en el servidor

La arquitectura genérica de un servidor con PHP se organiza en capas (Figura 2.1):

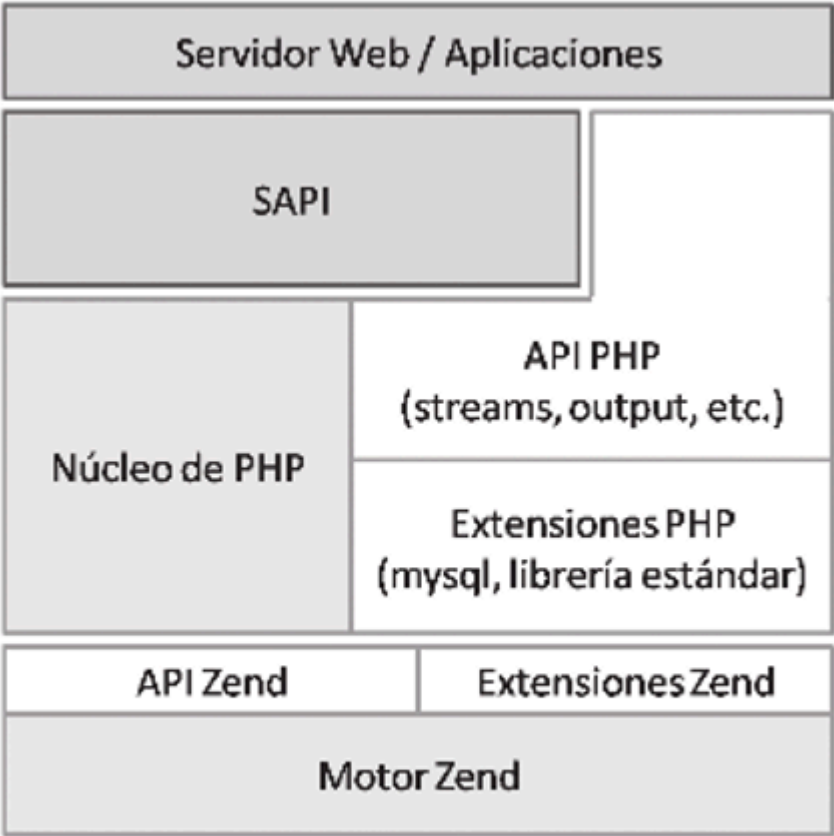


Figura 2.1. Arquitectura genérica de PHP con Zend

Capa	Función
Servidor web (capa más externa)	Recibe las peticiones del cliente a una dirección determinada.
SAPI (<i>Server Application Programming Interface</i>)	Es la interfaz mediante la cual PHP interactúa con el servidor u otras aplicaciones. Gestiona parcialmente el inicio y cierre de PHP e incluye mecanismos para acceder a datos del cliente, como las cookies o la información enviada por POST.
Núcleo de PHP	Maneja la configuración del entorno de ejecución (valores de variables globales y directivas de <code>php.ini</code>). Ofrece APIs de entrada/salida estándar, transformación de datos (<i>data parsing</i>) y una interfaz para cargar extensiones, tanto compiladas estáticamente como cargadas dinámicamente.

Capa	Función
Motor Zend (corazón de la arquitectura)	Analiza y ejecuta los scripts PHP. Admite extensiones que modifican toda su funcionalidad (compilación, ejecución, manejo de errores) o parte de ella.

Nota: la capa SAPI se llama oficialmente *Server API*. Hoy es habitual encontrar PHP como módulo de Apache o ejecutado mediante **PHP-FPM** (FastCGI Process Manager) detrás de Nginx o Apache.

2.3. Fases de ejecución de un script en el motor Zend

1. **Análisis léxico.** El analizador léxico (*lexer*) transforma el código escrito por el desarrollador en un conjunto de piezas o **tokens** que la máquina puede entender. Los tokens pasan al analizador sintáctico (*parser*).
2. **Compilación.** El *parser* toma los tokens y genera un conjunto de instrucciones o **código intermedio** (*opcodes*). Muchos *parsers* construyen un árbol sintáctico que puede manipularse u optimizarse antes de generar el código. El de Zend combina todos esos pasos en uno y produce el código intermedio directamente a partir de los tokens. Este proceso se denomina **compilación**.
3. **Ejecución.** El componente ejecutor de Zend recorre el código intermedio y ejecuta las instrucciones una por una.

Las fases de compilación y ejecución están implementadas como funciones accesibles desde el exterior, por lo que se pueden crear extensiones que modifiquen el comportamiento de cada una.

El motor Zend funciona como una **máquina virtual**, es decir, un programa que simula un ordenador físico. Implementa alrededor de 150 instrucciones específicas y optimizadas para PHP, como incluir librerías o imprimir una cadena.

Nota: en las versiones actuales, los *opcodes* pueden almacenarse en caché con **OPcache**, lo que evita repetir la compilación en cada petición.

3. Etiquetas para la inserción de código

En los lenguajes **embebidos**, la programación del lado del servidor se centra en insertar código propio dentro de páginas HTML. Cada lenguaje utiliza **etiquetas** que delimitan los fragmentos que debe procesar el servidor.

Sea cual sea el lenguaje (PHP, ASP, JSP u otros), el componente del servidor que procesa el código **ignora el HTML que queda fuera de esas etiquetas** y lo envía tal cual al cliente.

3.1. Ejemplo: «Hola mundo»

```
<html>
<head>
  <title>Primer ejemplo</title>
</head>
<body>
  <h1>Primer ejemplo: <br/></h1>
  <?php
```

```

        echo "Hola mundo.";
    ?>
</body>
</html>

```

Cuando el navegador solicita este recurso, el servidor ejecuta el fragmento PHP y envía al cliente el HTML resultante:

```

<html>
<head>
    <title>Primer ejemplo</title>
</head>
<body>
    <h1>Primer ejemplo: <br/></h1>
    Hola mundo.
</body>
</html>

```

El mismo resultado podría lograrse escribiendo el saludo directamente en el HTML. Sin embargo, este ejemplo sirve para introducir las características básicas de los lenguajes de código embebido.

3.2. Características del código embebido

- **Etiquetas de inicio y fin.** Todo script empieza y termina con una etiqueta (ver tabla siguiente).
- **Otros estilos de etiquetas.** Algunos lenguajes admiten configurar estilos alternativos.
- **Espacios en blanco.** Dentro del código embebido solo mejoran la legibilidad. Se admite cualquier combinación de espacios, tabuladores y saltos de línea entre sentencias.
- **Separación de sentencias.** Deben estar claramente separadas: en PHP y JSP con punto y coma (;), y en ASP con saltos de línea.
- **Posición libre.** Los scripts pueden situarse en cualquier parte del recurso y entre cualquier fragmento de HTML. El número de scripts por fichero es ilimitado.
- **Sustitución por el resultado.** Al ejecutarse, el script completo (incluidas las etiquetas de inicio y fin) se reemplaza por el resultado de su ejecución.

Lenguaje	Etiqueta de inicio	Etiqueta de fin	Observaciones
PHP	<?php	?>	Estilo estándar y recomendado.
PHP (abreviada)	<?	?>	<i>Short tags</i> : dependen de la directiva <code>short_open_tag</code> de <code>php.ini</code> . Se desaconseja su uso.
PHP (impresión)	<?=	?>	Equivale a <code><?php echo ... ?></code> . Está siempre disponible desde PHP 5.4.
ASP	<%	%>	<%= para imprimir un valor.
JSP	<%	%>	<%= para expresiones; <%! para declaraciones.

Actualización: el estilo `<script language="php"> ... </script>`, que existía en versiones antiguas de PHP, **se eliminó en PHP 7.0**. En ficheros que solo contienen PHP puede omitirse la etiqueta de cierre `?>` para evitar espacios en blanco accidentales al final del fichero.

3.3. Varios fragmentos de código en un mismo fichero

El siguiente ejemplo intercala tres fragmentos PHP. El primero define el contenido de la variable `$salida`. Los otros dos escriben ese valor en el título y en el cuerpo de la página.

```
<html>
<?php
    $salida = "Contenido PHP";
?>
<head>
    <title><?php echo $salida; ?></title>
</head>
<body>
    <h1><u>Segundo</u> ejemplo:</h1>
    <?php
        echo $salida;
    ?>
</body>
</html>
```

La posibilidad de situar scripts en cualquier parte del fichero es una ventaja, pero también **dificulta el mantenimiento** por la dispersión del código. Para reducir este efecto se usan directivas de inclusión de código, que se verán en capítulos posteriores (criterio RA2.f).

3.4. Inclusión de comentarios

Los comentarios explicativos son una buena práctica. Mejoran la legibilidad, aclaran las partes confusas y facilitan el mantenimiento y el trabajo en equipo.

```
// comentario de una línea
# otra forma de comentario de una línea

/* comentario
de varias
líneas */
```

También puede comentarse HTML, pero con diferencias importantes:

Tipo	Sintaxis	¿Llega al cliente?
Comentario PHP	<code>// ... , # ... o /* ... */</code>	No: lo elimina el servidor.
Comentario HTML	<code><!-- ... --></code>	Sí: cualquier usuario puede verlo en el código fuente.

Seguridad: no escribas datos sensibles (contraseñas, rutas internas) en comentarios HTML, porque son visibles para el cliente. Usa comentarios PHP.

3.5. Mostrar contenido en páginas HTML

La instrucción más habitual para mostrar contenido es **echo**, que sirve para cadenas y variables. La alternativa es **print**:

```
echo "ejemplo de impresión de cadena de caracteres";
print "ejemplo de impresión de cadena de caracteres"; // mismo resultado

echo 215062; // también se pueden imprimir números
echo $variableResultado; // y el contenido de una variable
print $variableResultado;
```

Una tercera forma, con la etiqueta de impresión:

```
<?= $variableResultado; ?>
```

Diferencias entre **echo** y **print**

Característica	echo	print
Tipo	Construcción del lenguaje	Construcción del lenguaje
Argumentos	Admite varios , separados por comas	Admite uno solo
Valor devuelto	Ninguno	Devuelve siempre 1
Rendimiento	Marginalmente más rápido	Marginalmente más lento

```
echo "primer argumento", " y el segundo"; // correcto
print "valor 1", "valor 2"; // INCORRECTO: print solo admite un
argumento
print "valor 1" . "valor 2"; // correcto: se usa el operador de
concatenación (.)
```

Precisión: aunque se suele decir que son funciones, **echo** y **print** son **construcciones del lenguaje** (*language constructs*). Por eso se pueden escribir sin paréntesis, aunque con ellos también funcionan: **echo ("mensaje");** equivale a **echo "mensaje";**. Con paréntesis, **echo** solo admite un argumento.

Ambas instrucciones muestran cadenas estáticas, números, variables y otros tipos. Los arrays no se imprimen directamente: para ellos se usa **print_r()** o **var_dump()**.

Comillas simples y dobles

Tanto las comillas simples (' ') como las dobles (" ") crean cadenas, pero se comportan de forma distinta:

	Comillas simples ' '	Comillas dobles " "
Interpreta variables	No	Sí
Interpreta secuencias de escape (\n, \t...)	Solo \' y \\\	Sí
Uso recomendado	Texto literal	Texto con variables o saltos de línea

```
$nombre = "Ana";  
echo 'Hola $nombre';    // muestra: Hola $nombre  
echo "Hola $nombre";    // muestra: Hola Ana
```

Para mostrar comillas dentro de la cadena hay dos opciones:

1. Usar el otro tipo de comilla como delimitador:

```
echo "This string has a ': a single quote!";  
echo 'This string has a ": a double quote';
```

2. «Escapar» el carácter con la barra invertida (\), que indica al intérprete que el carácter siguiente debe imprimirse sin interpretarlo:

```
echo "Este texto contiene una \" (una comilla doble);  
echo 'Este texto contiene una \' (una comilla simple)';
```

La barra invertida también sirve para insertar otros caracteres especiales en cadenas con comillas dobles:

```
echo "Este texto contiene una barra invertida: \\";  
echo "y este un símbolo del dólar: \$";
```

Otros lenguajes de servidor, como ASP o JSP, tienen sentencias similares que producen el mismo resultado.

Seguridad al mostrar contenido

Cuando se imprimen datos que proceden del usuario (formularios, URL, cookies) deben **escaparse** para evitar ataques **XSS** (*Cross-Site Scripting*):

```
echo htmlspecialchars($datoDelUsuario, ENT_QUOTES, 'UTF-8');
```

Esta función convierte caracteres como `<`, `>` y `"` en entidades HTML, de modo que el navegador los muestre como texto y no los ejecute como código.

4. Variables

Los datos que maneja un programa web se almacenan en **variables**: almacenes temporales que permiten gestionar la información durante la ejecución de una página. Toda variable tiene asociado un **tipo de datos**.

4.1. Definición y uso

En PHP las variables se identifican con el símbolo `$` seguido del nombre. No necesitan declararse de forma explícita y **no tienen tipo definido hasta que se les asigna un valor**.

```
$var = 15;    // $var es de tipo integer
```

En el momento de la primera asignación la variable se declara de forma implícita (se reserva memoria) y se fija su tipo. Esto es común a todas las variables PHP. El tipo puede **cambiar a lo largo de la vida de la variable**:

```
$var = 15;           // integer
$var = "cambio de tipo"; // ahora es string
```

Esta característica hace que PHP sea un lenguaje **loosely typed** (débilmente tipado): no exige declarar variables ni su tipo. Es más flexible, pero también más propenso a errores.

Actualización: PHP 7 y 8 permiten declarar tipos de forma opcional en parámetros, retornos y propiedades. Con `declare(strict_types=1);` se evita gran parte de las conversiones automáticas.

Reglas para los nombres de variable

Regla	Ejemplo válido	Ejemplo no válido
Empieza por una letra o un guión bajo (<code>_</code>).	<code>\$nombre,</code> <code>\$_dato</code>	<code>\$1dato</code>
Solo contiene caracteres alfanuméricos y guiones bajos (<code>a-z</code> , <code>A-Z</code> , <code>0-9</code> , <code>_</code>).	<code>\$mi_variable2</code>	<code>\$mi-variable</code>
No contiene espacios.	<code>\$miVariable</code>	<code>\$mi variable</code>

Para nombres de varias palabras se suele usar guión bajo (`$mi_variable`) o notación *camelCase* (`$miVariable`).

Ofuscación: existen técnicas de ofuscación de código, es decir, cambios no destructivos que dificultan su lectura (por ejemplo, para impedir la ingeniería inversa). Una forma básica es usar nombres de variable sin sentido. En proyectos propios debe hacerse lo contrario: usar **nombres descriptivos**.

PHP distingue mayúsculas de minúsculas

PHP es **case-sensitive** con los nombres de variable. Así, `$variable`, `$Variable` y `$VARIABLE` son tres variables distintas.

Un error en el nombre de una variable puede tener consecuencias graves, como un bucle infinito. En una web, esto haría que el navegador devolviera un error de *timeout* porque el servidor no termina de ejecutar la página.

```
for ($contador = 0; $contador < 10; $Contador++)  
    miFuncion();
```

En este código, `$contador` nunca se incrementa. Lo que se incrementa es otra variable distinta, `$Contador`, así que `$contador` siempre es menor que 10 y el bucle no termina.

Los errores más habituales son cambiar una letra, confundir mayúsculas y minúsculas, omitir guiones bajos o cometer erratas simples.

4.2. Tipos de datos

PHP permite incluir variables dentro de cadenas con comillas dobles. El intérprete sustituye el nombre de la variable por su valor en el momento de ejecutar la sentencia:

```
$hora = 12;  
$minutos = 15;  
$ciudad = "Sevilla";  
$texto = "El tren destino $ciudad sale a las $hora:$minutos";  
echo $texto;  
// La salida es: El tren destino Sevilla sale a las 12:15
```

Los tipos de PHP se agrupan en cuatro categorías:

Categoría	Tipos	Descripción
Escalares	<code>boolean</code> , <code>integer</code> , <code>float</code> , <code>string</code>	Contienen un único valor cada vez.
Compuestos	<code>array</code> , <code>object</code>	Formados por varios valores escalares u otros compuestos.
Especiales	<code>null</code> , <code>resource</code>	Valor nulo y referencias a recursos externos (ficheros, conexiones...).
Pseudotipos	<code>mixed</code> , <code>number</code> , <code>callback</code>	Se usan en la documentación para indicar que una función admite o devuelve valores de diferentes tipos.

Tipo boolean

Solo admite dos valores: `true` (cierto) y `false` (falso).

```
$repetidor = false;
$test = true;
```

Tipos integer y float

- **integer:** número entero, positivo o negativo.
- **float:** número real con parte entera y decimal (separadas por un **punto**). También puede escribirse en notación exponencial.

```
$num = 7;           // integer
$var2 = 5.12;       // float
$var3 = 1.234e4;     // float: representa 12340
$var4 = 4e-2;        // float: representa 0.04
```

Tipo string

Una cadena de caracteres se delimita con comillas simples o dobles (ver sección 3.5):

```
$variable = "Esto es una cadena de caracteres";
$test = 'Esto también es un string';
```

Constantes

Una **constante** es un valor que no cambia durante la ejecución. Se define con la función `define()`, que recibe dos argumentos obligatorios: el nombre y el valor.

```
define("PI", 3.14159);
echo PI;    // La salida es 3.14159
```

PHP ya incluye la constante `M_PI` y la función `pi()`:

```
<?php
echo pi();    // 3.1415926535898
echo M_PI;    // 3.1415926535898
?>
```

Características de las constantes:

- No llevan el símbolo `$` delante del nombre.
- Su valor **no puede cambiar** una vez definidas.
- Son accesibles desde cualquier parte del fichero PHP en ejecución.

- Pueden contener valores escalares y, desde PHP 7, también arrays.

Actualización: también se pueden definir con la palabra clave `const` (`const PI = 3.14159;`).

4.3. Asignación de valores

La asignación se hace con el signo `=`. Las expresiones numéricas funcionan como en otros lenguajes de alto nivel:

```
$var = 2 + 2;           // suma
$var = (($var - 3) * 4) / 2; // resta, multiplicación y división

// Tres formas de sumar 1
$var = $var + 1;
$var += 1;
$var++;

// Tres formas de restar 1
$var = $var - 1;
$var -= 1;
$var--;

// Duplicar y dividir por 2
$var = $var * 2;
$var *= 2;
$var = $var / 2;
$var /= 2;
```

Con las cadenas, el operador de **concatenación** es el punto (`.`), que también tiene versión abreviada (`.=`):

```
$var = "test string";           // asignar texto
$var = "cadena" . "unida";      // concatenar dos cadenas
$var = "cadena";
$var = $var . "unida";          // añadir texto al final
$var .= "test";                 // versión corta
```

4.4. Variables no inicializadas

Usar una variable antes de asignarle un valor **no detiene la ejecución**, pero se genera un aviso. En una expresión, la variable toma el valor por defecto de su tipo (`0` en operaciones numéricas, cadena vacía en concatenaciones).

```
<?php
$var1 = 100;
$var3 = $var1 + $var2; // $var2 no está inicializada
echo $var3;           // muestra 100

$var3 = 100 * $var2;
```

```
echo $var3;           // muestra 0
?>
```

Actualización: en PHP 8, usar una variable no definida genera un **Warning: Undefined variable** (en versiones anteriores era un **Notice**). En producción conviene no mostrar los errores al usuario y registrarlos en un fichero de log.

4.5. Conversiones entre tipos de datos

Lo que se muestra en el navegador siempre es **texto**: **echo** y **print** generan cadenas, porque un documento HTML se codifica en modo texto. Sin embargo, los lenguajes de alto nivel trabajan con distintos tipos de datos. La conversión entre tipos es habitual en entornos web, por ejemplo:

- Enviar una fecha no produce el mismo resultado que enviar un número real.
- Las operaciones más habituales exigen que los operandos sean del mismo tipo.

Funciones de conversión

Función	Resultado
<code>strval(mixed \$var)</code>	Convierte a string .
<code>intval(mixed \$var)</code>	Convierte a integer .
<code>floatval(mixed \$var)</code>	Convierte a float .
<code>settype(mixed &\$var, string \$tipo)</code>	Cambia el tipo de la propia variable. \$tipo puede ser array , boolean , float , integer , object o string .

Operadores de conversión (*casting*)

Como en el lenguaje C, se escribe el tipo deseado entre paréntesis delante de la variable.

Operador	Convierte a
<code>(int)</code> , <code>(integer)</code>	Entero
<code>(float)</code> , <code>(double)</code>	Número real
<code>(string)</code>	Cadena
<code>(bool)</code> , <code>(boolean)</code>	Booleano
<code>(array)</code>	Array
<code>(object)</code>	Objeto

```
$a = "123abc";
$b = (int) $a;    // 123
$c = (bool) "0";  // false
$d = (string) 3.5; // "3.5"
```

Las reglas de conversión (*cast rules*) son casi siempre de sentido común, pero algunas no son intuitivas:

Valor original	(int)	(bool)	(string)	(float)
"0"	0	false	"0"	0.0
"" (cadena vacía)	0	false	""	0.0
"abc"	0	true	"abc"	0.0
"12abc"	12	true	"12abc"	12.0
0	0	false	"0"	0.0
1.9	1	true	"1.9"	1.9
null	0	false	""	0.0
[] (array vacío)	0	false	"Array" (con aviso)	0.0
true	1	true	"1"	1.0
false	0	false	""	0.0

Importante: al convertir un número real a entero, PHP **trunca** la parte decimal; no redondea.

Conversión automática (*type juggling*)

Cuando una expresión combina variables de distinto tipo, o una función recibe un tipo distinto del esperado, PHP convierte el valor automáticamente al tipo requerido.

```
$var = "20" + 15;      // integer: 35
$var = "20" + 15.1;    // float: 35.1
$var = 20 . " años";   // string: "20 años"
```

Algunas conversiones no son tan obvias y son origen de errores difíciles de detectar:

```
$var = 40 + "2 razones";      // integer 42, con aviso (cadena numérica solo al
                               // inicio)
$var = 20 + " años";          // PHP 7: 20 con aviso. PHP 8: lanza TypeError
$var = "prueba" * 4 + 3.14159; // PHP 7: 3.14159 con aviso. PHP 8: lanza
                               // TypeError
```

Diferencia entre versiones: en PHP 8, usar en una operación aritmética una cadena **no numérica** lanza una excepción `TypeError`. En PHP 7 solo generaba un aviso. Fíjate también en la diferencia entre `.` (concatena) y `+` (suma).

La conversión automática también puede cambiar el tipo de una variable:

```
$var = "1";    // string "1"
$var += 2;     // integer 3
$var /= 2;     // float 1.5
$var *= 2;     // float 3.0
```

Valores no booleanos usados como booleanos

Cuidado con los valores no booleanos que se tratan como **boolean**. Algunas funciones de la librería estándar devuelven **false** cuando no pueden devolver un resultado. Los siguientes valores se interpretan **siempre como false**:

- 0 y 0.0
- "0" y la cadena vacía ""
- null
- un array vacío []

La solución es **comprobar siempre el tipo** del valor devuelto, con los operadores de comparación estricta (===, !==) o con estas funciones:

Función	Devuelve true si la variable es...
is_int(\$var)	un entero
is_float(\$var)	un número real
is_bool(\$var)	un booleano
is_string(\$var)	una cadena
is_array(\$var)	un array
is_object(\$var)	un objeto

```
$prueba = 13.0;
echo is_float($prueba);    // imprime 1 (true)
```

Mostrar el tipo y el valor de una variable

Para depurar se usan **var_dump()** y **print_r()**, que muestran el tipo y el valor actual:

```
$variable = 15;
var_dump($variable);    // La salida es: int(15)
```

4.6. Precedencia de los operadores

La precedencia en PHP es similar a la de otros lenguajes: la multiplicación y la división se evalúan antes que la suma y la resta. Depender de ella con frecuencia produce código confuso y propenso a errores.

Recomendación: usar **paréntesis**, que tienen la máxima precedencia, para dejar claro el orden de las operaciones.

```
$operacion = 2 + 5 * 6;      // 32: la multiplicación se evalúa primero
$operacion = 2 + (5 * 6);    // 32: el mismo resultado, pero más claro
$operacion = (2 + 5) * 6;    // 42: los paréntesis cambian el orden
```

4.7. Estado de una variable

PHP ofrece tres funciones para comprobar el estado de una variable, sea cual sea su tipo: **isset()**, **is_null()** y **empty()**. Todas devuelven **true** o **false**. Muchas veces son intercambiables, pero en algunos casos dan resultados inesperados.

A menudo se piensa que **is_null()** es lo contrario de **isset()**, y esta lo contrario de **empty()**. Eso no es del todo cierto.

is_null()

Comprueba si una variable es **NULL**, un valor especial que representa una variable sin valor. Devuelve **true** cuando la variable:

1. No ha sido definida.
2. Está definida pero sin valor asignado.
3. Tiene asignado el valor **NULL**.
4. Ha sido destruida con **unset()**.

```
var_dump(is_null($var));    // no definida: true, y lanza un aviso de variable no
                             definida

$var = NULL;
var_dump(is_null($var));    // true

$var;
var_dump(is_null($var));    // «declarada» sin valor
                             // true

$var = "Hola";
unset($var);
var_dump(is_null($var));    // true, y lanza un aviso de variable no definida
```

Con una variable no definida (casos 1 y 4), **is_null()** genera un aviso. **isset()** y **empty()** **no lo generan**.

isset()

Según el manual de PHP, determina si una variable **está declarada y su valor no es NULL**.

- Una variable con valor **NULL** hace que **isset()** devuelva **false**, aunque esté declarada.
- Con valores vacíos, como una cadena vacía, devuelve **true**.

```
var_dump(isset($var));    // false: $var no está definida

$var;
var_dump(isset($var));    // false: su valor es NULL

$var = "";
var_dump(isset($var));    // true: el valor ya no es nulo, aunque esté vacío
```

empty()

Determina si una variable **está vacía**. Se considera vacía si no existe, o si su valor equivale a **false**. Esto incluye:

- el booleano **false**,
- la cadena vacía **""**,
- el número **0**,
- el array vacío **[]**,
- **NULL**,
- y, como peculiaridad, el **string "0"**.

```
var_dump(empty($var));    // no definida: true

$var;
var_dump(empty($var));    // true

$var = "";    var_dump(empty($var));    // true
$var = array();    var_dump(empty($var));    // true
$var = "0";    var_dump(empty($var));    // true (el string "0" cuenta como vacío)
$var = 0;    var_dump(empty($var));    // true
$var = false;    var_dump(empty($var));    // true
$var = true;    var_dump(empty($var));    // false
```

Tabla comparativa

Valor de \$var	isset(\$var)	is_null(\$var)	empty(\$var)	(bool)\$var
(no definida)	false	true (con aviso)	true	false (con aviso)
NULL	false	true	true	false
""	true	false	true	false
"0"	true	false	true	false

Valor de <code>\$var</code>	<code>isset(\$var)</code>	<code>is_null(\$var)</code>	<code>empty(\$var)</code>	<code>(bool)\$var</code>
<code>0</code>	<code>true</code>	<code>false</code>	<code>true</code>	<code>false</code>
<code>false</code>	<code>true</code>	<code>false</code>	<code>true</code>	<code>false</code>
<code>[]</code>	<code>true</code>	<code>false</code>	<code>true</code>	<code>false</code>
<code>"hola"</code>	<code>true</code>	<code>false</code>	<code>false</code>	<code>true</code>
<code>1</code>	<code>true</code>	<code>false</code>	<code>false</code>	<code>true</code>
<code>true</code>	<code>true</code>	<code>false</code>	<code>false</code>	<code>true</code>

Destruir una variable con `unset()`

La función `unset()` destruye una variable, que queda «no definida»:

```
$var = "valor";
unset($var);

if (isset($var)) echo "var está definida";    // no imprime nada
```

Equivalencia entre `(bool)` y `!empty()`

Forzar el tipo `boolean` con `(bool)` equivale a comprobar que la variable no está vacía con `!empty()`:

```
$var = "valor";

if ((bool) $var) echo "var no está vacía";
if (!empty($var)) echo "var no está vacía";    // ambas imprimen lo mismo
```

¿Cuándo utilizar cada función?

No hay una respuesta única, pero estas pautas sirven:

Función	Úsala cuando...
<code>empty()</code>	Es la más frecuente. Quieres comprobar que la variable no tiene un valor vacío , sin que importe si estaba declarada (si no existe devuelve <code>true</code> sin lanzar error). No la uses si <code>"0"</code> o <code>0</code> son valores válidos.
<code>isset()</code>	Necesitas comprobar que la variable existe y no es nula , aunque su valor sea vacío.
<code>is_null()</code>	Casos poco frecuentes: comprobar específicamente que la variable es nula o no existe.
Comparación estricta (<code>===</code>)	Si no sabes si la variable está declarada, compruébalo antes con <code>isset()</code> y después compara.

Alternativa moderna: el operador de fusión de *null* `??` (PHP 7+) simplifica estas comprobaciones:

```
$nombre = $_POST['nombre'] ?? 'Anónimo';
```

4.8. Ámbito de una variable

El **ámbito** (*scope*) es el contexto en el que una variable está definida, es decir, la zona del programa donde puede accederse a ella. Hay ámbito **local** y **global**. Además existe la variable de **sesión**, que se verá en un tema posterior.

Ámbito local

Las variables usadas dentro de una función **son distintas de las que se usan fuera** de ella. Las variables internas de una función solo pueden utilizarse dentro de dicha función.

```
function duplicar($var) {  
    $temp = $var * 2;  
}  
  
$variable = 5;  
duplicar($variable);  
echo "El valor de la variable \">$temp es: $temp";
```

Salida:

```
El valor de la variable $temp es:
```

La variable `$temp` es **local** a `duplicar()` y se destruye al salir de la función. Fuera de ella, `$temp` no existe.

PHP no detiene la ejecución por esto. Trata la variable como vacía. Es posible configurar el intérprete para que muestre un aviso con las variables no declaradas (en PHP 8 se muestra un **warning** si el nivel de errores lo permite).

Devolver el valor con **return**

La forma más sencilla de usar fuera un valor local de una función es **devolverlo**:

```
function duplicar($var) {  
    $resultado = $var * 2;  
    return $resultado;  
}  
  
$variable = 5;  
$temp = duplicar($variable);  
echo "El valor de la variable \">$temp es: $temp";
```

Salida:

```
El valor de la variable $temp es: 10
```

Podría haberse usado el nombre `$temp` dentro de la función, pero sería una variable distinta de la que recoge el valor devuelto. **Regla general:** las variables usadas solo dentro de una función son locales, aunque exista fuera otra con el mismo nombre.

Existe una excepción: las variables definidas **por referencia** y las declaradas como **globales** dentro de la función no son locales (se estudiarán más adelante).

Ámbito global con `global`

Para usar la misma variable en cualquier parte del código, incluso dentro de funciones, se emplea la sentencia `global`. Declara que la variable de la función es la misma que se usa fuera.

```
function duplicar() {  
    global $temp;  
    $temp = $temp * 2;  
}  
  
$temp = 5;  
duplicar();  
echo "El valor de la variable $temp es: $temp";
```

Salida:

```
El valor de la variable $temp es: 10
```

Buenas prácticas: abusar de las variables globales dificulta el mantenimiento y favorece los errores. Se recomienda **pasar los datos por parámetros y devolverlos con `return`**.

Diferencias entre lenguajes

Lenguaje	Cómo se declara una variable global
PHP	Dentro de la función donde se va a usar (<code>global \$var;</code>).
ASP y JSP	En el programa principal, fuera del ámbito de la función.

5. Buenas prácticas y seguridad (ampliación)

Estos puntos complementan el currículo (RA2) con hábitos que se evaluarán durante el curso:

- **Separar en lo posible lógica y presentación.** Evita mezclar grandes bloques de PHP con HTML; usa ficheros de inclusión (`include`, `require`). Esto se estudiará con más detalle en los temas siguientes.
- **Usar nombres descriptivos** en variables y funciones, y comentar el código.

- **Comprobar siempre el tipo** de los valores que devuelven las funciones y los datos que llegan del cliente.
- **Escapar toda salida** que contenga datos del usuario (`htmlspecialchars()`).
- **No mostrar errores en producción.** Configura `display_errors = Off` y registra los errores en un fichero de log.
- **Mantener PHP actualizado.** Las versiones sin soporte no reciben parches de seguridad.
- **Cumplir la normativa de protección de datos.** Si la aplicación trata datos personales (formularios, cookies, registros de usuarios), debe respetar el **Reglamento General de Protección de Datos (RGPD, UE 2016/679)** y la **Ley Orgánica 3/2018 (LOPDGDD)**. Para cookies y servicios de la sociedad de la información, la **Ley 34/2002 (LSSI-CE)**.

Resumen del tema

Concepto	Idea clave
Servidor web	Recibe peticiones HTTP, ejecuta el código y devuelve HTML.
HTTP	Protocolo sin estado; HTTP/1.1 añade conexiones persistentes.
Tipos de servidor	Procesos, hilos, eventos y núcleo del sistema.
Servidores populares	Apache, IIS, iPlanet, Nginx, Lighttpd (y Tomcat como servidor de aplicaciones).
PHP	Lenguaje embebido; arquitectura en capas (servidor, SAPI, núcleo, motor Zend).
Etiquetas	<code><?php ... ?></code> en PHP; el HTML fuera de las etiquetas se envía tal cual.
Salida	<code>echo</code> (varios argumentos) y <code>print</code> (uno, devuelve 1); <code><?= ?></code> abreviado.
Comillas	Simples = literal; dobles = interpretan variables y escapes.
Variables	Empiezan por <code>\$</code> , distinguen mayúsculas y tienen tipado débil.
Tipos	Escalares, compuestos, especiales y pseudotipos.
Conversión	Explícita (<code>(int)</code> , <code>intval()</code> ...) o automática (<i>type juggling</i>).
Estado	<code>isset()</code> , <code>is_null()</code> , <code>empty()</code> : cada una con sus peculiaridades.
Ámbito	Local por defecto; <code>global</code> o <code>return</code> para compartir valores.